



CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

PROYECTO INTEGRADOR:

Sistema de Gestión de Pruebas Integrales para Aplicación Web
Aplicación de Estándares, Métricas y Herramientas Modernas de QA/QC

Oscar Mauricio Areiza Paramo (9023125050),
Facultad De Ingeniería, Corporación Universitaria del Huila (Corhuila)
Ingeniería De Sistemas (Calidad de Software 2026A)
Phd. José Migue Llanos
14 mayo 2026

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación
y extensión de programas de pregrado y posgrado, en sus
sedes Neiva y Pitalito"

Tabla de Contenido

Introducción	4
Objetivos	5
Objetivo General	5
Objetivos Específicos	5
• Implementar pruebas unitarias	5
• Auditar la interfaz de usuario (UI/UX):	5
• Evaluar el rendimiento del backend:	5
• Garantizar la calidad estática del código	5
• Documentar la estrategia de calidad:	5
• Automatizar la infraestructura y despliegue:	5
Desarrollo	6
Alcance del Proyecto, Casos de Uso y Herramientas	6
Fase 1:	7
Fase 2	10
Fase 3	13
Fase 4:	17
Fase 5:	22
Fase 6:	23
Resultados Obtenidos	24
Integración y Funcionamiento de la API REST	24
Cobertura de Código y Pruebas Unitarias	24
Calidad de Código y Análisis Estático	25
Rendimiento y Experiencia de Usuario en el Frontend.....	26
Pruebas de Carga y Rendimiento.....	26
Conclusiones	28
Referencias	29



Tabla de ilustraciones

Ilustración 1 Tablero de desarrollo – Trello.....	7
Ilustración 2 Archivo docker-compose.yml.....	8
Ilustración 3 Contenedor-e-imágenes-SGP_LAB.....	8
Ilustración 4 Paquete Springboot y Dependencias	9
Ilustración 5 Vinculación Del Proyecto Al Contenedor	9
Ilustración 6 POMP.XML.....	10
Ilustración 7 Diagrama Entidad Relación (DER)	10
Ilustración 8 Creación de tablas	11
Ilustración 9 Lógica De Estados(enum)	11
Ilustración 10 Creación De Los Repositorios	12
Ilustración 11 Creación De La Interfaz De Los Servicios	12
Ilustración 12 Creación De Los Servicios	13
Ilustración 13 Construcción De Los Controladores	13
Ilustración 14 Postman	14
Ilustración 15 Pruebas Unitarias – Controladores	15
Ilustración 16 Pruebas Unitarias - Servicios	15
Ilustración 17 Análisis Con SonarQube:	16
Ilustración 18 Errores Criticos.....	16
Ilustración 19 Otros Errores	17
Ilustración 20 Login	18
Ilustración 21 Vista Cliente/Admin.....	18
Ilustración 22 Creación de Servicios API.....	19
Ilustración 23 Creación De Rutas.....	19
Ilustración 24 Definición de los Temas Y Plugins	20
Ilustración 25 Creación De Los Componentes	20
Ilustración 26 Diagnostico google Lighthouse.....	21
Ilustración 27 Error Critico Login.....	21
Ilustración 28 Jenkins.....	23
Ilustración 29 Autenticaciones En Postman	24
Ilustración 30 Resultados Maventest – JACOCO.....	25
Ilustración 31 Diagnostico SonarQube V2.....	25
Ilustración 32 Lighthouse después de mejoras	26
Ilustración 33 Prueba de carga.....	27
Ilustración 34 Pruebas de estres	27
Ilustración 35 Resultados Jmeter	28





Introducción

El presente reporte documenta el proceso integral de desarrollo, aseguramiento y control de calidad (QA/QC) del Sistema de Gestión de Préstamos de Laboratorio (SGPLab), proyecto final de la asignatura de Calidad de Software. En el ciclo de vida del desarrollo de software moderno, la calidad no es una fase final, sino un atributo transversal que debe integrarse desde la concepción de la arquitectura hasta su despliegue en producción.

Bajo esta premisa, el SGPLab fue construido utilizando una arquitectura de tres capas desarrollada en Java con Spring Boot, garantizando la separación de responsabilidades, la seguridad de la información y la integridad de los datos. Para validar el cumplimiento de los estándares internacionales como la norma ISO/IEC 25010 y la ISO/IEC 29119, se diseñó e implementó un ecosistema de pruebas automatizadas. Este ecosistema abarca desde pruebas unitarias con JUnit, análisis de calidad estático con SonarQube, hasta auditorías de rendimiento y experiencia de usuario (UX) en el frontend mediante Google Lighthouse, culminando con pruebas de carga y estrés empleando Apache JMeter. Todo este flujo se encuentra orquestado en contenedores Docker y un pipeline de Integración y Despliegue Continuo (CI/CD) soportado por GitHub Actions, demostrando la aplicación práctica de metodologías ágiles y herramientas de vanguardia para la entrega de un producto robusto, seguro y eficiente.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Objetivos

Objetivo General

Aplicar de forma integral los conceptos teóricos de calidad de software, los estándares internacionales (ISO/IEC 25010, ISO/IEC 29119) y las herramientas modernas de QA/QC en el diseño, desarrollo, validación y automatización del Sistema de Gestión de Préstamos de Laboratorio (SGPLab) utilizando un entorno tecnológico basado en Java y contenedores Docker.

Objetivos Específicos

- **Implementar pruebas unitarias:** Desarrollar una suite de pruebas con JUnit orientada a la capa de lógica de negocio, asegurando una cobertura de código mínima del 80% verificada mediante JaCoCo.
- **Auditar la interfaz de usuario (UI/UX):** Ejecutar pruebas de rendimiento, accesibilidad y buenas prácticas en el frontend de la aplicación web utilizando Google Lighthouse, garantizando una experiencia de usuario óptima.
- **Evaluar el rendimiento del backend:** Diseñar y ejecutar pruebas de carga, estrés y capacidad con Apache JMeter para identificar tiempos de respuesta, *throughput* y posibles cuellos de botella en la API REST.
- **Garantizar la calidad estática del código:** Integrar SonarQube en el ciclo de desarrollo para el análisis continuo del código fuente, reportando y mitigando vulnerabilidades, *code smells* y deuda técnica.
- **Documentar la estrategia de calidad:** Elaborar un plan de pruebas estructurado y trazable bajo los lineamientos del estándar ISO/IEC 29119.
- **Automatizar la infraestructura y despliegue:** Orquestar los servicios de la aplicación, bases de datos y herramientas de calidad mediante Docker Compose, implementando un pipeline de CI/CD con GitHub Actions para la ejecución automática de las pruebas.





Desarrollo

Alcance del Proyecto, Casos de Uso y Herramientas

El Sistema de Gestión de Préstamos de Laboratorio (SGP_LAB) se concibió como una aplicación web enfocada en administrar el inventario y controlar el flujo de préstamos de equipos hacia los estudiantes de la universidad. El alcance del proyecto, aunque de naturaleza simple, se centró en garantizar la integridad de los datos y gestionar de manera estricta las reglas de negocio y las transiciones de estados (disponible, prestado, en mantenimiento, penalizado). La aplicación soporta dos perfiles principales: el Administrador, con permisos de gestión y control total, y el Cliente, orientado a la solicitud y consulta de equipos.

Para estructurar el desarrollo, se definieron los siguientes Casos de Uso (CU) e Historias de Usuario (HU):

- **CU/HU Autenticación y Seguridad:** Acceso diferenciado al sistema mediante validación de credenciales y roles (Administrador o Cliente).
- **CU/HU CRUD de Inventario:** Gestión integral de los equipos físicos del laboratorio, previniendo duplicidad de registros.
- **CU/HU Búsqueda y Filtros:** Consulta de disponibilidad de equipos en tiempo real para facilitar la toma de decisiones del usuario.
- **CU/HU Solicitud de Préstamo:** Creación de peticiones de préstamo validando disponibilidad de fechas y límites de uso por estudiante.
- **CU/HU Aprobación/Rechazo:** Control administrativo para autorizar o denegar préstamos, cambiando el estado lógico del equipo.
- **CU/HU Devolución y Verificación:** Retorno del equipo al inventario disponible, registrando su estado físico tras el uso.
- **CU/HU Gestión de Mantenimiento:** Aislamiento preventivo o correctivo de un equipo, bloqueando temporalmente su solicitud.





- **CU/HU Penalizaciones y Bloqueos:** Mecanismo automático de sanción para usuarios que incumplen los plazos de devolución.
- **CU/HU Reporte de Uso:** Generación de métricas de utilización para apoyar el mantenimiento predictivo.

Para llevar a cabo la construcción y validación del sistema, se seleccionó un conjunto de herramientas alineadas con los estándares de calidad de la industria. Se utilizó **Java** y **Spring Boot** para el backend, junto con bases de datos relacionales (**PostgreSQL** y **H2**). La orquestación y el despliegue se gestionaron mediante **Docker**, **Docker Compose** y **GitHub Actions**. Las pruebas y auditorías de calidad se realizaron utilizando **JUnit**, **Postman**, **Google Lighthouse**, **Apache JMeter**, **JaCoCo** y **SonarQube**, **Jenkins**. Toda la planificación del proyecto se estructuró a través de un tablero de trabajo en **Trello**, dividiendo la ejecución en seis fases lógicas de desarrollo, como se observa en la ilustración 1.



Ilustración 1 Tablero de desarrollo – Trello

Enlace: <https://trello.com/invite/b/6a05f59a15cd791725eca15c/ATTI6ad012111ffb5d4ccfbad0e2d708a11aB31511D5/sgplab>

Fase 1: Infraestructura y Entorno En esta etapa inicial se estableció el ecosistema donde operaría la aplicación. Se configuraron los contenedores para la base de datos, el backend y las herramientas de análisis de código. Asimismo, se implementaron las rutinas de integración continua en el repositorio, garantizando que el entorno de despliegue fuera consistente y reproducible desde el primer día.



En la **Ilustración [2]** se observa la **construcción de Docker Compose**, sobre el cual se desplegarán los servidores para la base de datos **PostgreSQL**, **Jenkins**, **JMeter** y demás métricas contempladas.

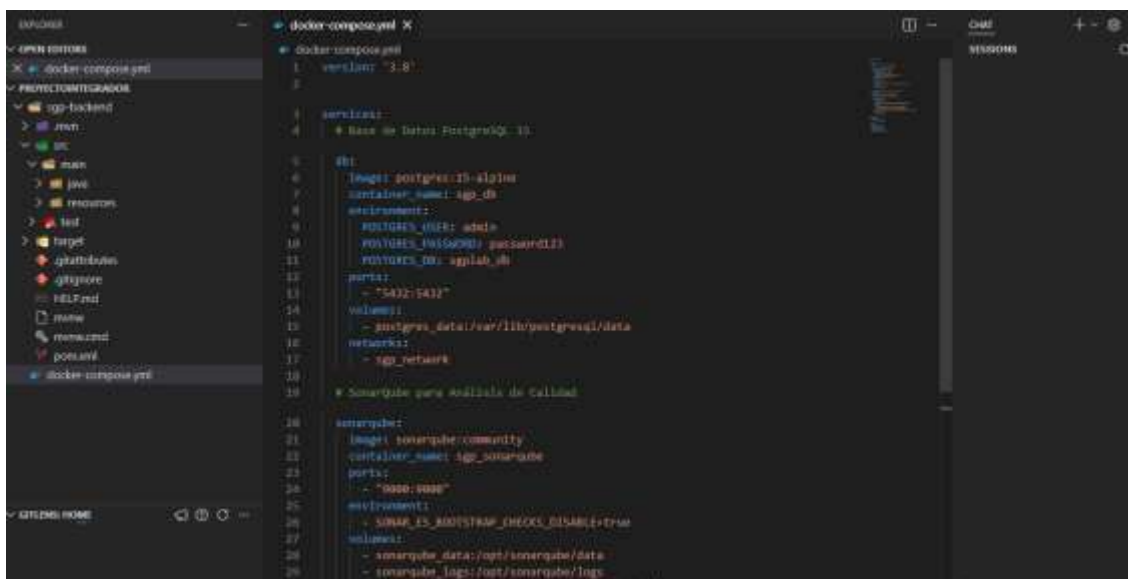


Ilustración 2 Archivo docker-compose.yml

En la **Ilustración [3]** se observan los servidores de Docker Compose ya construidos, sobre los cuales se trabajará y se iniciará el desarrollo de **SGP_LAB**.

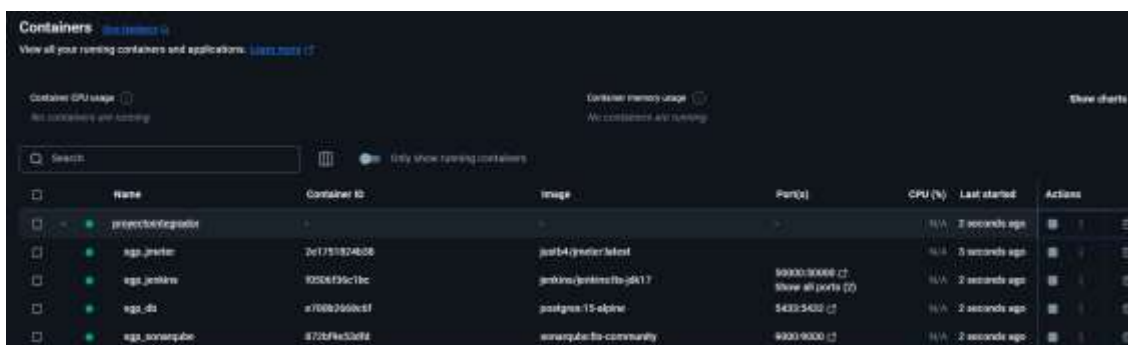


Ilustración 3 Contenedor-e-imágenes-SGP_LAB

En la Ilustración [4] se visualiza la **construcción** del paquete de **Spring Boot** y sus respectivas **dependencias** para el manejo de **JPA**, **ORM**, **DevTools**, **Spring Test**, entre otras.

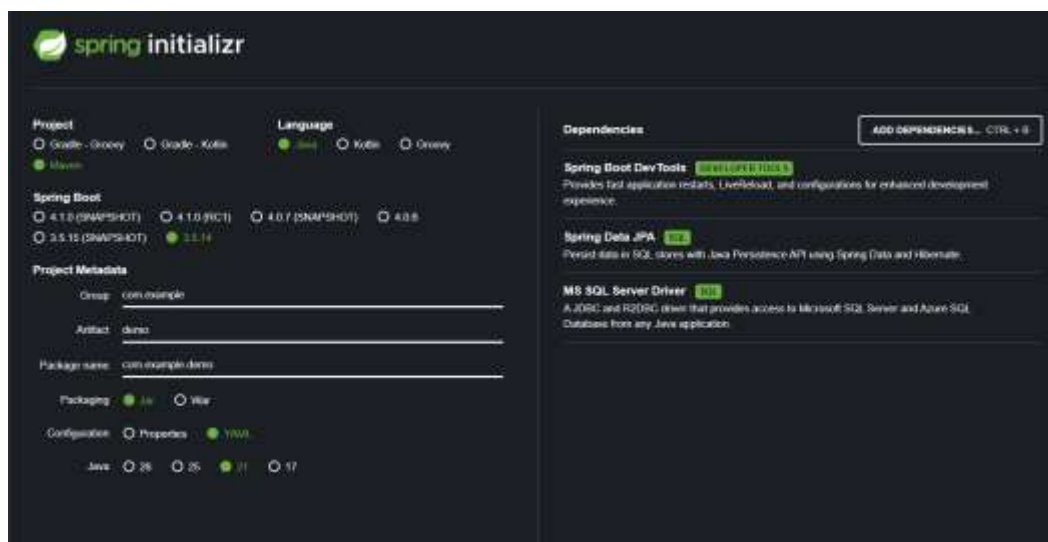


Ilustración 4 Paquete Springboot y Dependencias

En la **Ilustración [5]** se observa la construcción del documento que vincula el proyecto con el contenedor de **Docker**.

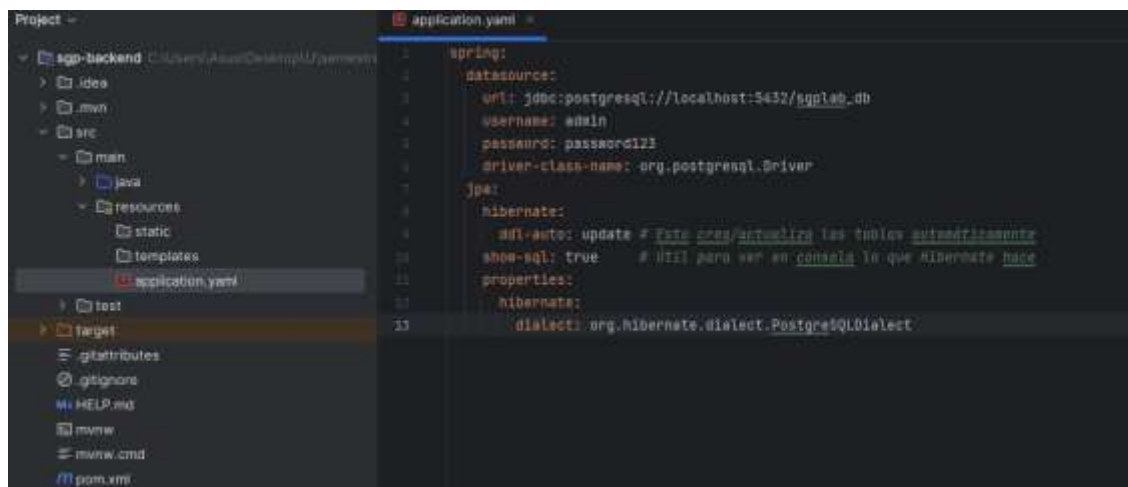


Ilustración 5 Vinculación Del Proyecto Al Contenedor



En la Ilustración [6] se muestra el archivo **pom.xml**, donde se encuentran las diferentes dependencias, plugin y configuraciones para el desarrollo del proyecto.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2003/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.1.5-SNAPSHOT</version>
        <relativePath></relativePath> </parent>
    <groupId>com.eggs</groupId>
    <artifactId>eggs-backend</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>eggs-backend</name>
    <description>SGP Backend con Integración SonarQube</description>

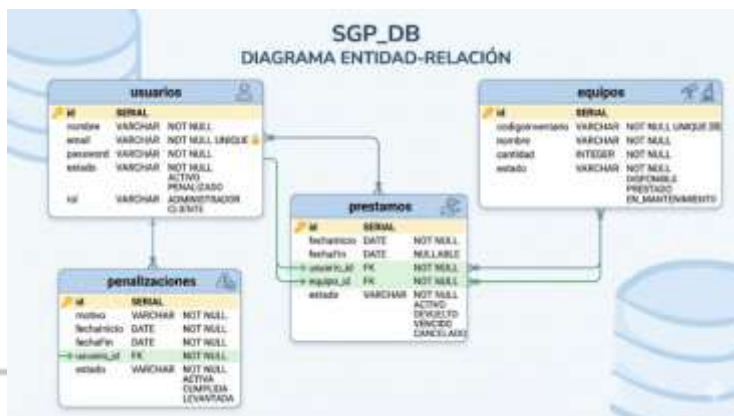
    <properties>
        <jdk.version>11</jdk.version>
        <sonar.java.binDir>target/classes</sonar.java.binDir>
        <sonar.coverage.jacoco.xmlReportPaths>${project.build.directory}/site/jacoco/jacoco.xml</sonar.coverage.jacoco.xmlReportPaths>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
    </dependencies>
```

Ilustración 6 POMP.XML

Fase 2: Diseño de Datos y Seguridad Durante esta fase, el trabajo se enfocó en la capa de persistencia. Se estructuraron las entidades de la base de datos y sus relaciones, aplicando restricciones estrictas a nivel de esquema para garantizar la integridad de la información. A la par, se implementó la capa de seguridad, integrando los mecanismos de cifrado de contraseñas y la validación de tokens de acceso para distinguir las operaciones permitidas según el rol del usuario.

Ilustración 7 Diagrama Entidad Relación (DER)



Sede Quirinal: Calle 21 No. 6 - 01

Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



En la **Ilustración [8]** aparece el script de la entidad **Usuario**, que es una de las tablas construidas a partir del **modelo conceptual** visto en la **Ilustración [7]**. Se realizó el mismo proceso para las demás entidades, tales como **Préstamo**, **Penalización** y **Equipo**.

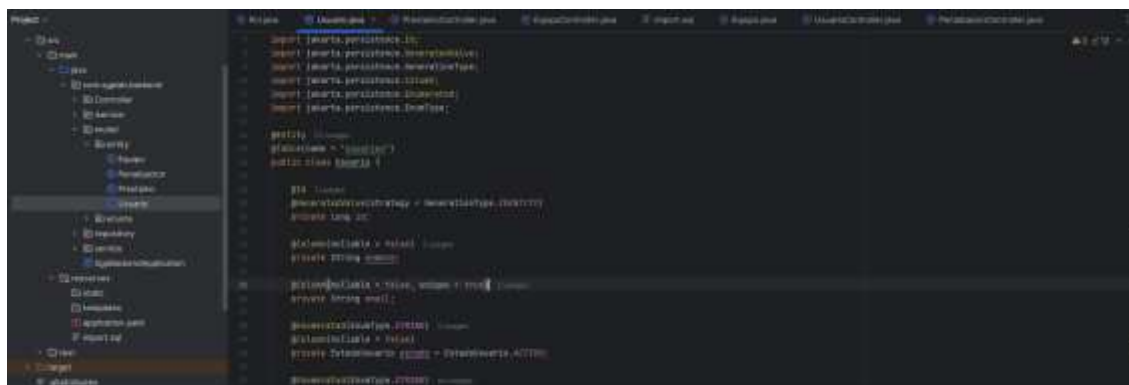


Ilustración 8 Creación de tablas

En la **Ilustración [9]** se observa la lógica bajo la cual se construirán los estados de los diferentes atributos en las tablas, utilizando tipos enumerados (**Enums**)



Ilustración 9 Lógica De Estados(enum)

En la **Ilustración [10]** se observa el resultado de la **construcción** de las interfaces **IRepository**, las cuales cumplen la función de gestionar el acceso a los datos dentro de la arquitectura del proyecto.

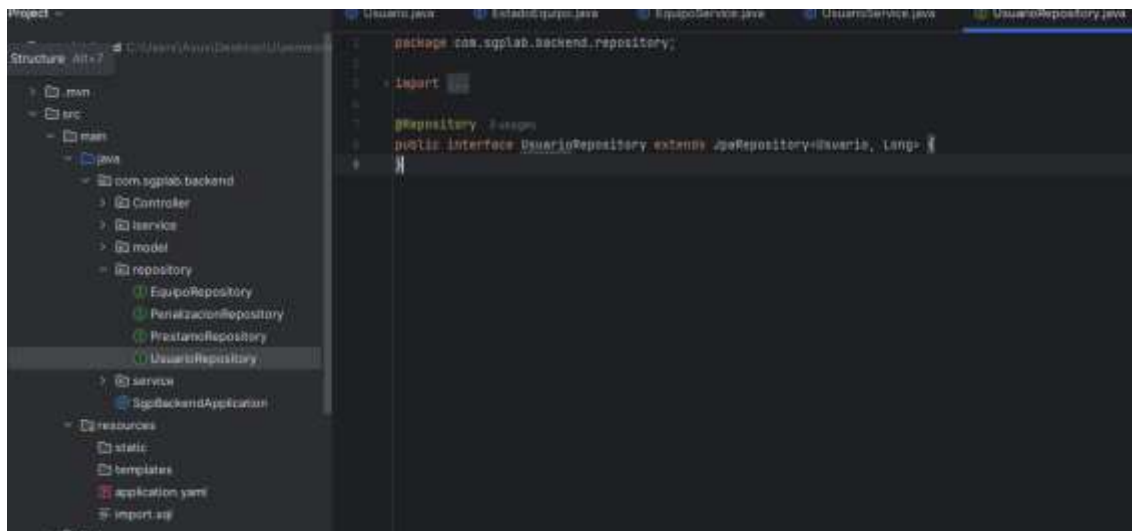


Ilustración 10 Creación De Los Repositorios

En la **Ilustración [11]** se detalla la **construcción** de las **interfaces de los servicios** para cada entidad del proyecto, las cuales definen los contratos de la lógica de negocio.

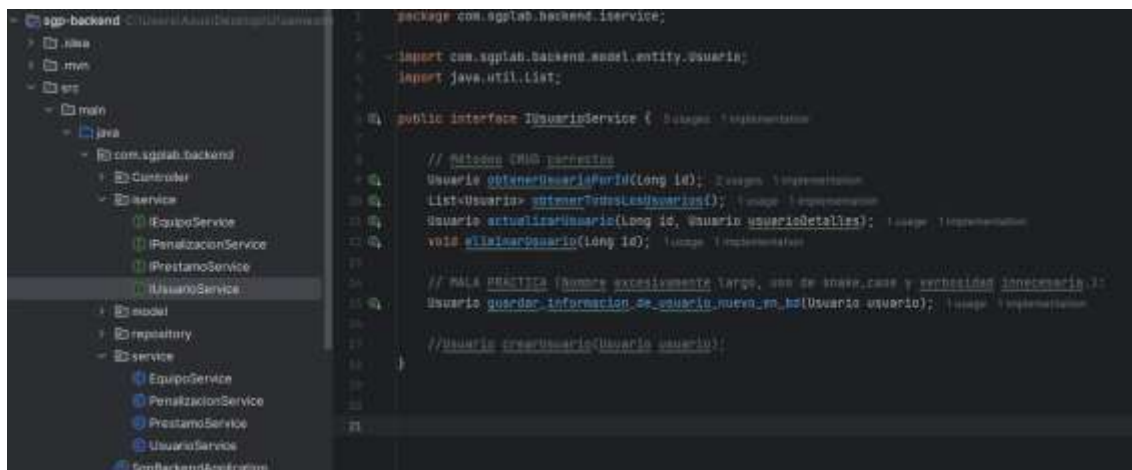


Ilustración 11 Creación De La Interfaz De Los Servicios

En la **Ilustración [12]** se detalla la **creación de los servicios**, los cuales contienen la implementación de los **métodos propios** y la lógica de negocio específica para cada clase del sistema.

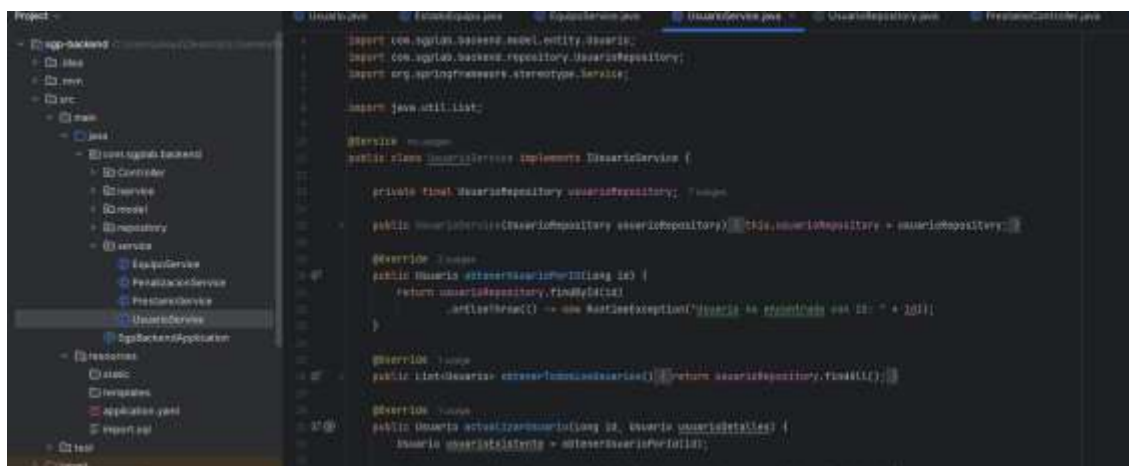


Ilustración 12 Creación De Los Servicios

Fase 3 Lógica de Negocio y API REST. En esta etapa se construyó el núcleo funcional de la aplicación. Se desarrollaron los servicios encargados de ejecutar las reglas de negocio, gestionar los estados de los equipos y procesar las penalizaciones; posteriormente, se expuso esta lógica mediante la construcción de la **API REST**, tal como se observa en la **Ilustración [13]**.



Ilustración 13 Construcción De Los Controladores

Para asegurar que la comunicación entre la API y la base de datos operara correctamente, se realizaron pruebas exhaustivas utilizando Postman. Estas pruebas permitieron validar el envío de datos, la recepción de códigos de estado adecuados y el correcto funcionamiento de los filtros de seguridad.

En la **Ilustración [14]** se observa la creación de una **colección de peticiones en Postman**, donde se encuentran los métodos **GET, POST, PUT y DELETE** para cada una de las entidades existentes en el proyecto.

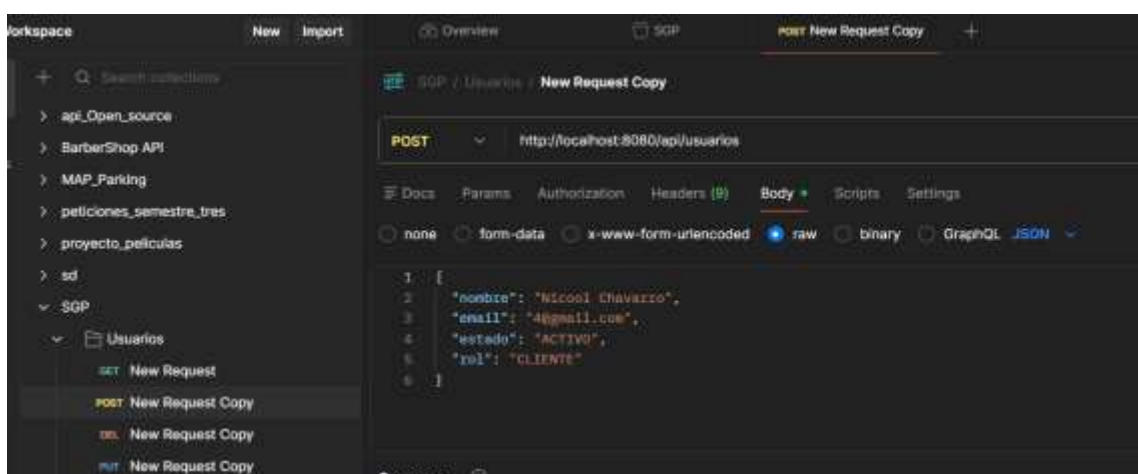


Ilustración 14 Postman

Simultáneamente a la construcción del código, se desarrollaron las pruebas unitarias para validar la lógica de negocio de manera aislada, evaluando tanto los flujos exitosos como el manejo de excepciones, logrando así las métricas de cobertura exigidas.

En el marco de la evaluación de calidad de software, se desarrollaron pruebas unitarias e integrales (**Ilustraciones [15] y [16]**) bajo un esquema de experimentación controlada. En el caso de las pruebas de integración, se introdujeron fallos deliberados con el fin de alterar los indicadores de rendimiento. Esta estrategia busca simular un entorno de baja calidad para



observar el incremento en la **densidad de defectos** y la disminución del **porcentaje de cobertura**, cumpliendo con los criterios de evaluación establecidos para el proyecto.



Ilustración 15 Pruebas Unitarias – Controladores

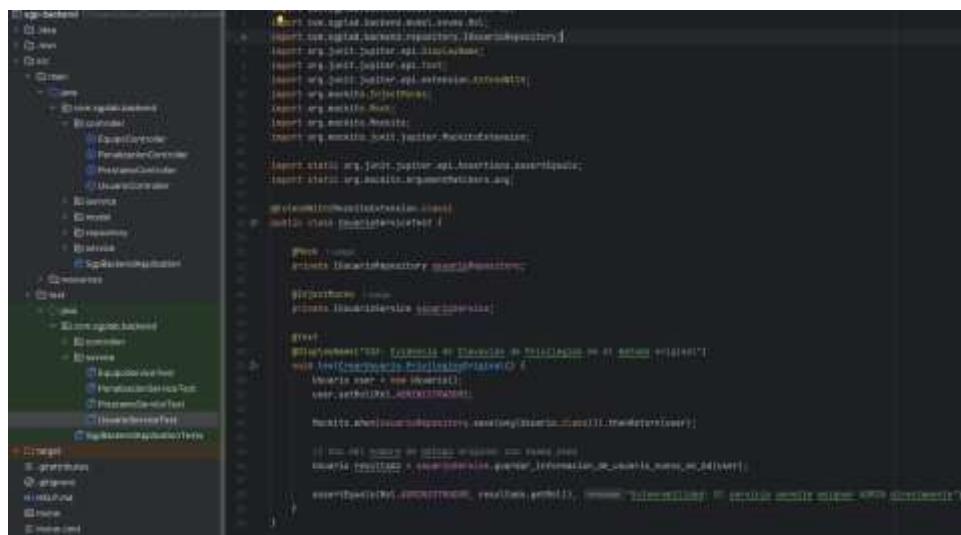


Ilustración 16 Pruebas Unitarias - Servicios



En la **Ilustración [17]** se visualiza la plataforma **SonarQube** con los resultados generales del análisis, mientras que en la **Ilustración [18]** se detallan los diversos errores y vulnerabilidades detectados durante la ejecución del proceso.

Con el código base construido, se procedió a aplicar las métricas de calidad. Tras ejecutar los análisis estáticos de código con SonarQube, se obtuvo el siguiente diagnóstico detallando vulnerabilidades, código duplicado y deudas técnicas.



Ilustración 17 Análisis Con SonarQube:



Ilustración 18 Errores Criticos



A diferencia de las ilustraciones anteriores, donde se evidencian fallos críticos como el manejo inadecuado de excepciones, brechas de seguridad y problemas de infraestructura, en la **Ilustración [19]** se observan errores de menor impacto. Entre ellos destacan problemas de nomenclatura de variables, comentarios fuera de lugar y diversos **Code Smells**, que representan oportunidades de mejora en la legibilidad y mantenimiento del código.

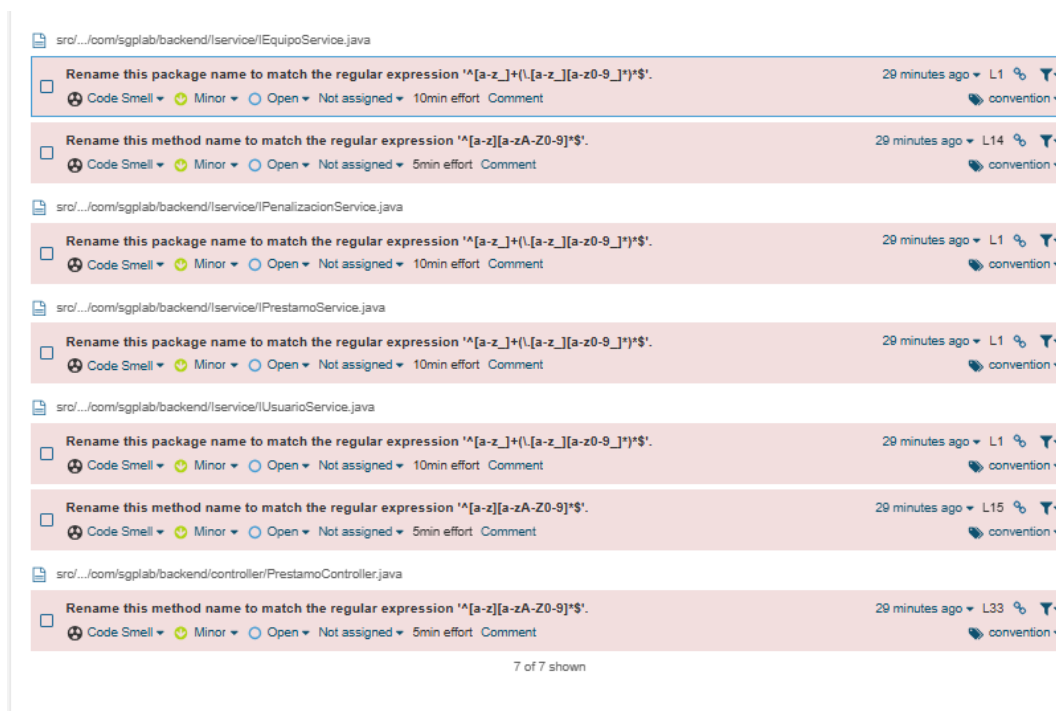


Ilustración 19 Otros Errores

Fase 4: Frontend y Experiencia de Usuario Una vez estabilizado el backend, se procedió al desarrollo de la interfaz gráfica. Se construyeron los formularios y paneles de control, consumiendo los servicios de la API REST previamente probados y asegurando que los mensajes de error y validaciones se mostraran claramente al usuario.

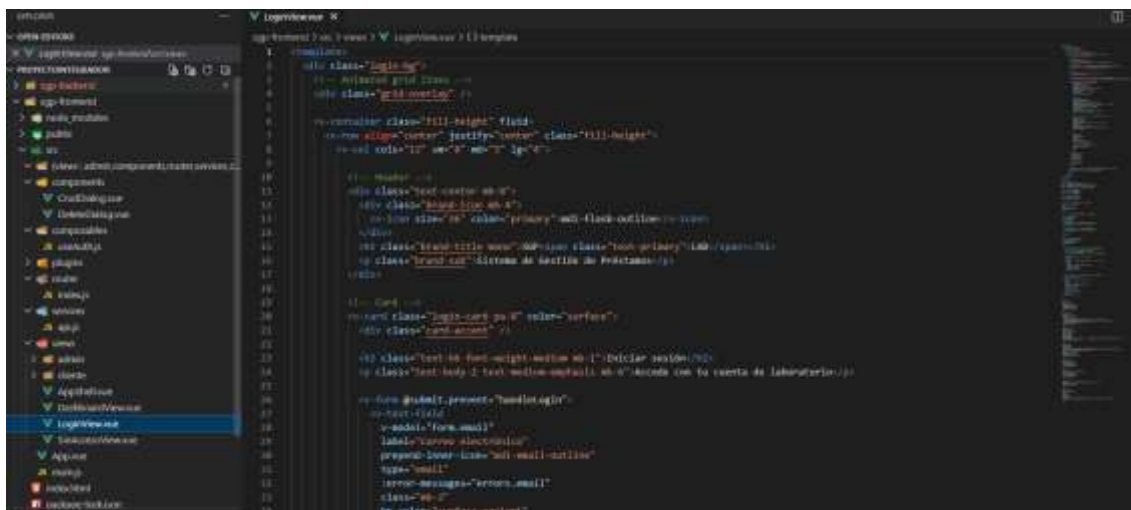


Ilustración 20 Login

En la **Ilustración [20]** se observa la construcción del **login** con diversas fallas de seguridad, tales como la inexistencia de un proceso de **autenticación de contraseñas** y la posibilidad de elegir arbitrariamente el acceso entre la vista de cliente o la de administrador **Ilustración [21]**

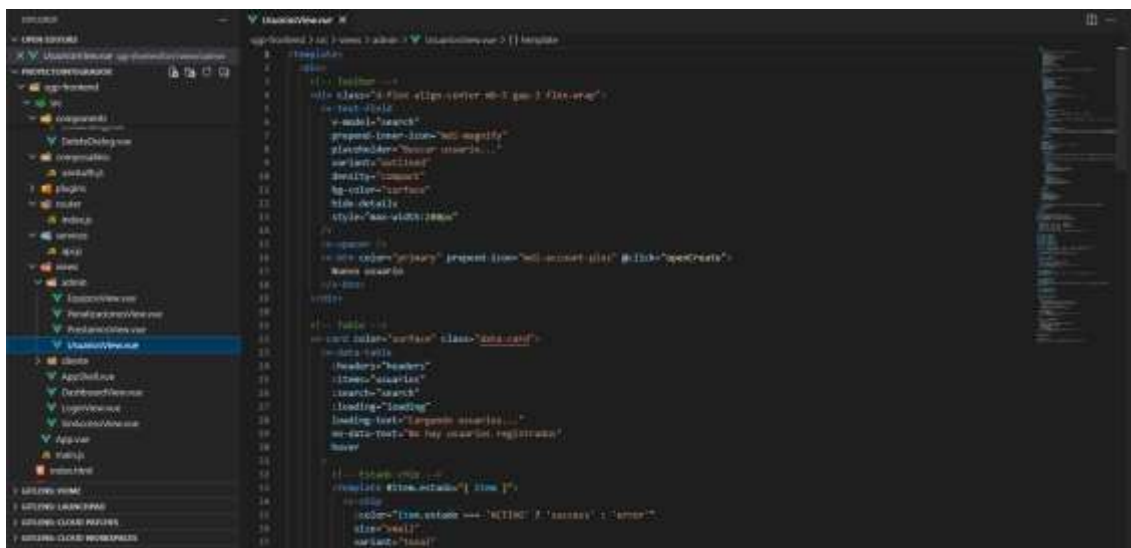


Ilustración 21 Vista Cliente/Admin

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"

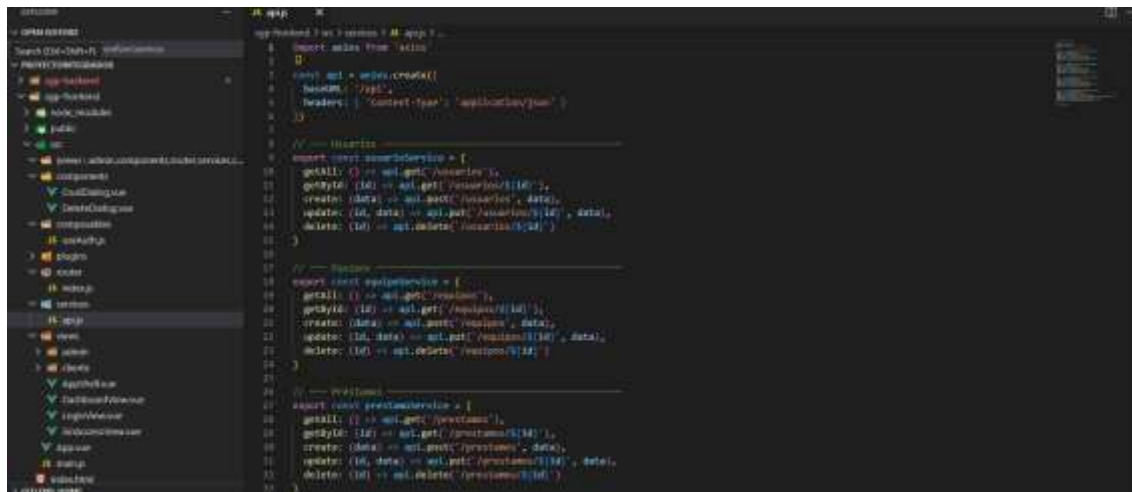


Ilustración 22 Creación de Servicios API

En la **Ilustración [22]** se observa la **construcción de los servicios de la API**, los cuales permiten la comunicación y el intercambio de datos entre los diferentes componentes del sistema.



Ilustración 23 Creación De Rutas

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"

En la **Ilustración [23]** se definen la configuración visual y estructural del **frontend**, especificando los **temas**, las hojas de **estilo** y los **plugins** necesarios para garantizar una interfaz de usuario coherente y funcional.

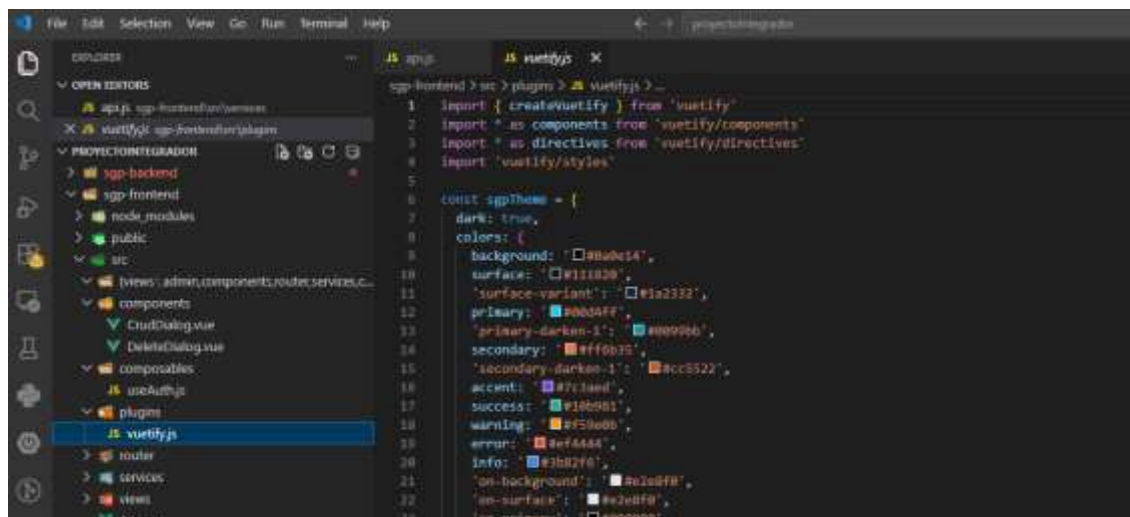


Ilustración 24 Definición de los Temas Y Plugins

En la **Ilustración [24]** se detalla la **construcción de los componentes del frontend**, los cuales consisten en piezas de código independientes y reutilizables que encapsulan la lógica y la interfaz de usuario para facilitar el mantenimiento y la escalabilidad del proyecto.



Ilustración 25 Creación De Los Componentes

Para evaluar la calidad de la capa de presentación, se ejecutaron auditorías automatizadas utilizando Google Lighthouse. Esto permitió medir el rendimiento, la accesibilidad y el cumplimiento de buenas prácticas web de la interfaz.



Ilustración 26 Diagnostico google Lighthouse

En la **Ilustración [25]** se presentan los resultados del diagnóstico inicial del **frontend** mediante la herramienta **Google Lighthouse**. El reporte evidencia puntuaciones significativamente bajas en las métricas de rendimiento y accesibilidad, debido principalmente a la implementación de malas prácticas de desarrollo y a la falta de optimización en la interfaz de usuario (UI).



Ilustración 27 Error Critico Login



En la **Ilustración [26]** se detallan las deficiencias críticas identificadas en el diagnóstico de **Google Lighthouse**, sumadas a una vulnerabilidad grave en la lógica de navegación. Se evidencia que el sistema no requiere validación de credenciales, permitiendo el ingreso arbitrario a cualquier rol; de este modo, un usuario puede acceder indistintamente a la interfaz de cliente o de administrador mediante una selección manual, sin que exista una restricción basada en privilegios.

Fase 5: Aseguramiento de Calidad y Rendimiento

En esta fase se evaluó el comportamiento del sistema bajo condiciones de estrés mediante el diseño de escenarios de prueba con Apache JMeter para simular solicitudes concurrentes de múltiples usuarios hacia la API. Sin embargo, las pruebas de estrés no pudieron ejecutarse satisfactoriamente debido a múltiples problemas críticos identificados tanto en el backend como en el frontend del sistema.

A nivel de backend, se detectó que el login estaba completamente simulado sin autenticación real, las contraseñas se almacenaban en texto plano sin ningún tipo de cifrado, los endpoints carecían de protección mediante tokens JWT, las entidades JPA se exponían directamente en las respuestas REST incluyendo campos sensibles como el hash de contraseña, no existían DTOs lo que generaba acoplamiento total entre la capa de persistencia y la capa REST, cualquier error del servidor devolvía un código HTTP 500 genérico por la ausencia de un manejador global de excepciones, las validaciones de datos se realizaban de forma manual e insuficiente, los tests unitarios estaban escritos intencionalmente para fallar y la cobertura de código era inferior al 20%.

A nivel de frontend, el login no conectaba con el backend y solo simulaba la autenticación mediante un selector de rol, no existía un interceptor para enviar el token JWT en las peticiones, no había manejo de expiración de sesión ni redirección automática al login, los servicios del API presentaban inconsistencias en los nombres de métodos que provocaban errores en tiempo de ejecución y los views de administración fallaban al intentar cargar datos.

Ante estos hallazgos, se propone aplicar las correcciones necesarias en ambas capas del sistema y posteriormente reejecutar la suite completa de pruebas de estrés y carga para verificar la estabilidad y el rendimiento del aplicativo bajo condiciones de uso real.



Recomendación: Automatización del ciclo de pruebas con Jenkins

Tras aplicar las correcciones identificadas en la fase anterior, se recomienda implementar un pipeline de integración continua mediante Jenkins que permita automatizar la ejecución completa del ciclo de pruebas cada vez que se realice un cambio en el código fuente. De esta manera, las pruebas unitarias, el análisis de cobertura con JaCoCo, el escaneo de calidad con SonarQube y las pruebas de carga con JMeter se ejecutarían de forma automática y secuencial, garantizando que cualquier nueva modificación no introduzca regresiones ni comprometa la estabilidad del sistema. Esta práctica asegura un control de calidad continuo y reduce significativamente el riesgo de desplegar código con defectos a producción.



Ilustración 28 Jenkins

Fase 6: Documentación e Informe Final La etapa final consistió en la consolidación de los artefactos del proyecto. Se documentaron los planes de prueba y se recopilaron todas las métricas de cobertura y calidad generadas por las distintas herramientas a lo largo de las fases previas, asegurando el cumplimiento de la norma y cerrando el ciclo de vida del desarrollo del SGP_LAB.

Con el objetivo de dar cumplimiento a los requerimientos establecidos, se procedió con la construcción y desarrollo de los diferentes documentos y entregables solicitados. Este proceso se ejecutó de manera sistemática, asegurando la consolidación de toda la información y el alineamiento con los objetivos del proyecto.



Resultados Obtenidos

Tras concluir el ciclo iterativo de desarrollo y aplicar las refactorizaciones y correcciones correspondientes en cada una de las fases, se procedió a la reejecución completa de la suite de pruebas y auditorías de calidad. El objetivo de esta fase final fue verificar el impacto de las mejoras implementadas y asegurar el cumplimiento de los estándares de calidad definidos al inicio del proyecto. Los resultados obtenidos en esta última iteración demostraron una mejora significativa en todas las métricas evaluadas, consolidando la estabilidad, seguridad y eficiencia del sistema.

Integración y Funcionamiento de la API REST

Al ejecutar nuevamente las peticiones hacia los servicios del backend, se comprobó que los flujos de negocio operan de manera correcta. El manejo de roles, la validación de estados y la gestión de penalizaciones responden con los códigos de estado HTTP esperados, demostrando una comunicación robusta y segura entre la interfaz, la API y la base de datos.

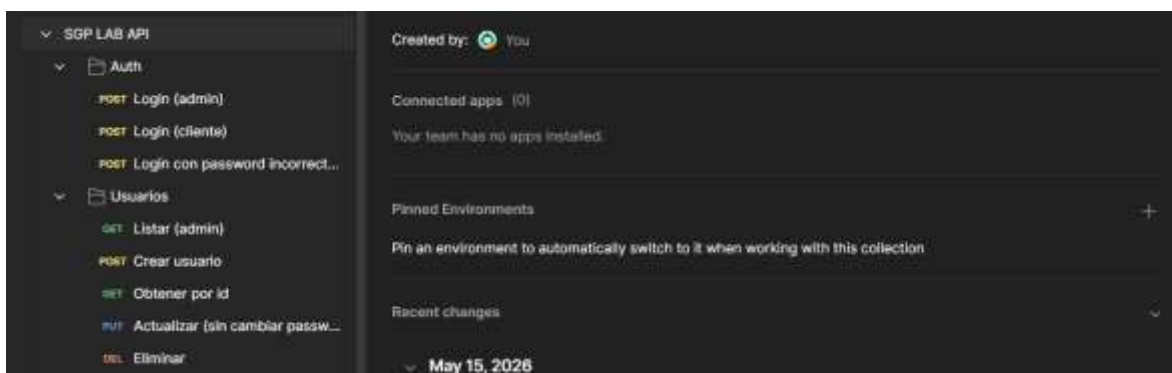


Ilustración 29 Autenticaciones En Postman

Cobertura de Código y Pruebas Unitarias

La ejecución final de la suite de pruebas unitarias confirmó que la lógica de negocio se encuentra protegida ante escenarios exitosos y de error. Las métricas de cobertura superaron el umbral mínimo establecido del 80%, asegurando que las reglas centrales del sistema fueron probadas exhaustivamente.





```
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.278 s -- in com.sgplab.backend.unitarias.servi
ce.UsuarioServiceImplTest
[INFO] Running com.sgplab.backend.unitarias.util.PasswordHashUtilTest
[INFO] Running com.sgplab.backend.unitarias.util.PasswordHashUtilTest$HashTests
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.059 s -- in com.sgplab.backend.unitarias.util.P
asswordHashUtilTest$HashTests
[INFO] Running com.sgplab.backend.unitarias.util.PasswordHashUtilTest$MatchesTests
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.065 s -- in com.sgplab.backend.unitarias.util.P
asswordHashUtilTest$MatchesTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.109 s -- in com.sgplab.backend.unitarias.util.P
asswordHashUtilTest
[ERROR] Surefire is going to kill self fork JVM. The exit has elapsed 30 seconds after System.exit(0).
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 114, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ sgp-backend ---
[INFO] Loading execution data file /app/target/jacoco.exec
[INFO] Analyzed bundle 'sgp-backend' with 46 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 06:15 min
[INFO] Finished at: 2026-05-15T16:41:26Z
[INFO]
C:\Users\Asus\Desktop\CS_PI\CORHUILA\2026A\calidad-software\talleres\tercer-corte\sgp-v2\proyecto_integrador_v2\sgp-back
```

Ilustración 30 Resultados Maventest – JACOPO

Calidad de Código y Análisis Estático

El escaneo final del código fuente reflejó una disminución drástica en la deuda técnica. Se mitigaron las vulnerabilidades de seguridad, se eliminó el código duplicado y se corrigieron los code smells, obteniendo calificaciones óptimas que garantizan la mantenibilidad del software a largo plazo.

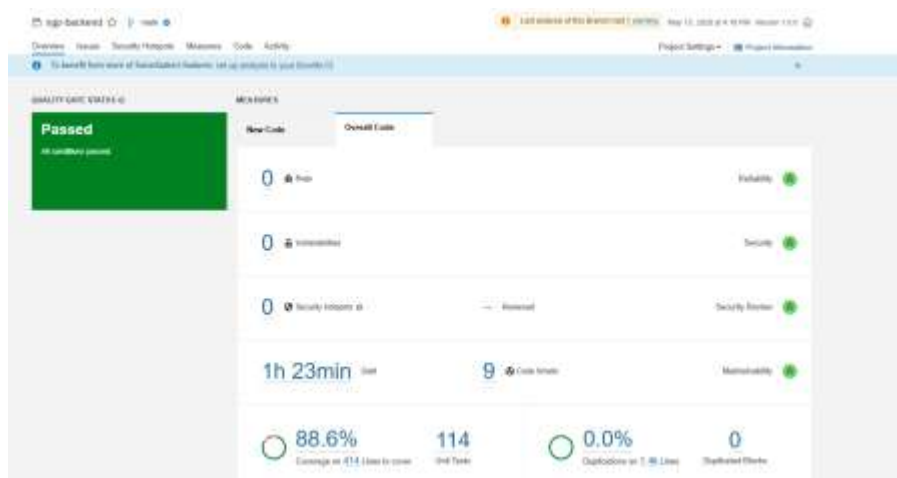


Ilustración 31 Diagnóstico SonarQube V2

- 📍 Sede Quirinal: Calle 21 No. 6 - 01
- 📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
- 📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
- ✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Rendimiento y Experiencia de Usuario en el Frontend

Las auditorías finales de la interfaz gráfica evidenciaron un incremento notable en las puntuaciones generales. Las optimizaciones aplicadas mejoraron los tiempos de carga, el cumplimiento de criterios de accesibilidad y la adherencia a las mejores prácticas de desarrollo web.

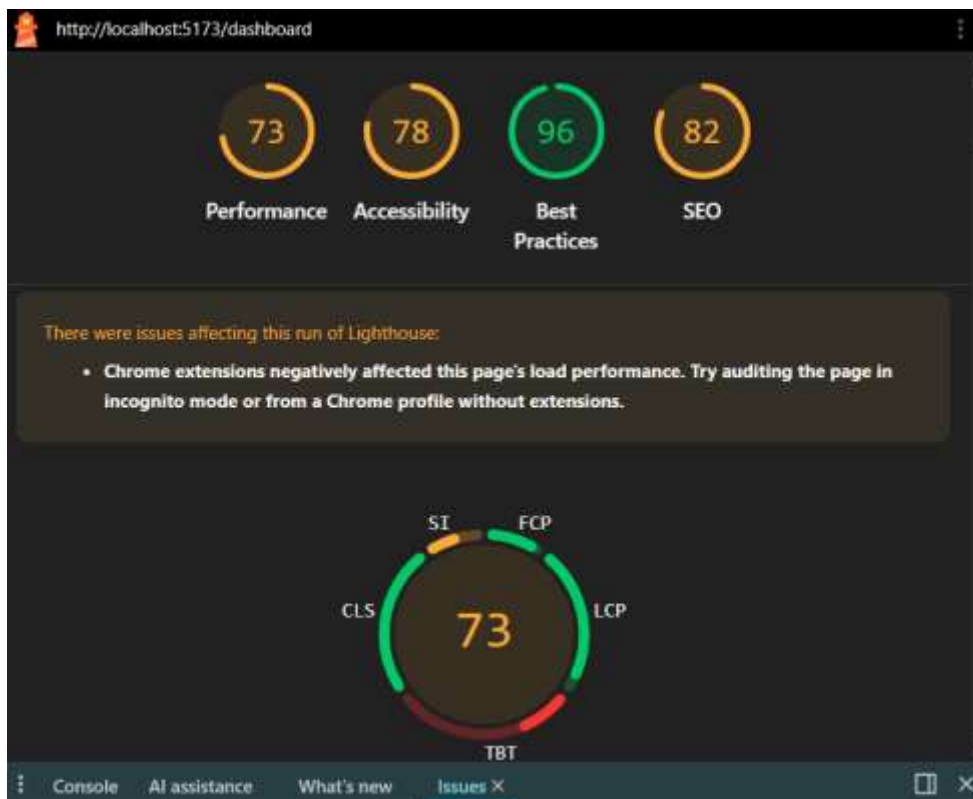


Ilustración 32 Lighthouse después de mejoras

Pruebas de Carga y Rendimiento

Tras aplicar las mejoras en el manejo de recursos, la ejecución de los escenarios de estrés bajo concurrencia mostró un sistema más resiliente. Se observó una estabilización en los tiempos de respuesta y una tasa de error mínima, comprobando que la aplicación es capaz de soportar la carga esperada por los usuarios del laboratorio.



```
Starting standalone test @ May 15, 2026 11:44:40 PM CEST (177888148824)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
Summary + 162 in 00:00:17 = 9.3/s Avg: 258 Min: 11 Max: 5065 Err: 8 (4.94%) Active: 28 Started: 28 Finis
d: 0
Summary + 1229 in 00:00:30 = 41.0/s Avg: 54 Min: 3 Max: 1276 Err: 1 (0.08%) Active: 50 Started: 50 Finis
d: 0
Summary = 1391 in 00:00:47 = 29.4/s Avg: 77 Min: 3 Max: 5065 Err: 9 (0.65%)
Summary + 629 in 00:00:14 = 44.7/s Avg: 25 Min: 3 Max: 452 Err: 0 (0.00%) Active: 0 Started: 50 Finish
: 50
Summary = 2020 in 00:01:01 = 32.9/s Avg: 61 Min: 3 Max: 5065 Err: 9 (0.45%)
Dying up ... @ May 15, 2026 11:45:44 PM CEST (1778881544294)
... end of run
ND Running Jmeter on Fri May 15 23:46:04 CEST 2026
```

Ilustración 33 Prueba de carga

```
Creating summariser <summary>
Created the tree successfully using estres.jmx
Starting standalone test @ May 15, 2026 11:49:58 PM CEST (1778881798837)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
Summary + 1 in 00:00:06 = 0.2/s Avg: 949 Min: 949 Max: 949 Err: 0 (0.00%) Active: 13 Started: 13 Finis
hed: 0
Summary + 2401 in 00:00:25 = 96.0/s Avg: 62 Min: 2 Max: 1563 Err: 0 (0.00%) Active: 99 Started: 99 Finis
hed: 0
Summary = 2482 in 00:00:31 = 77.5/s Avg: 62 Min: 2 Max: 1563 Err: 0 (0.00%)
Summary + 4037 in 00:00:30 = 135.0/s Avg: 40 Min: 2 Max: 826 Err: 0 (0.00%) Active: 200 Started: 200 Fin
shed: 0
Summary = 6439 in 00:01:01 = 105.7/s Avg: 48 Min: 2 Max: 1563 Err: 0 (0.00%)
Summary + 4617 in 00:00:32 = 144.2/s Avg: 68 Min: 2 Max: 3480 Err: 0 (0.00%) Active: 200 Started: 200 Fin
shed: 0
Summary = 11056 in 00:01:33 = 118.9/s Avg: 56 Min: 2 Max: 3480 Err: 0 (0.00%)
Summary + 1535 in 00:00:28 = 55.1/s Avg: 194 Min: 1 Max: 7415 Err: 0 (0.00%) Active: 200 Started: 200 Fin
shed: 0
Summary = 12591 in 00:02:01 = 104.2/s Avg: 73 Min: 1 Max: 7415 Err: 0 (0.00%)
Summary + 31 in 00:00:13 = 2.4/s Avg: 173 Min: 18 Max: 841 Err: 0 (0.00%) Active: 0 Started: 200 Finis
hed: 200
Summary = 12622 in 00:02:14 = 94.4/s Avg: 73 Min: 1 Max: 7415 Err: 0 (0.00%)
Dying up ... @ May 15, 2026 11:52:14 PM CEST (1778881934238)
... end of run
ND Running Jmeter on Fri May 15 23:52:32 CEST 2026
```

Ilustración 34 Pruebas de estrés



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2829

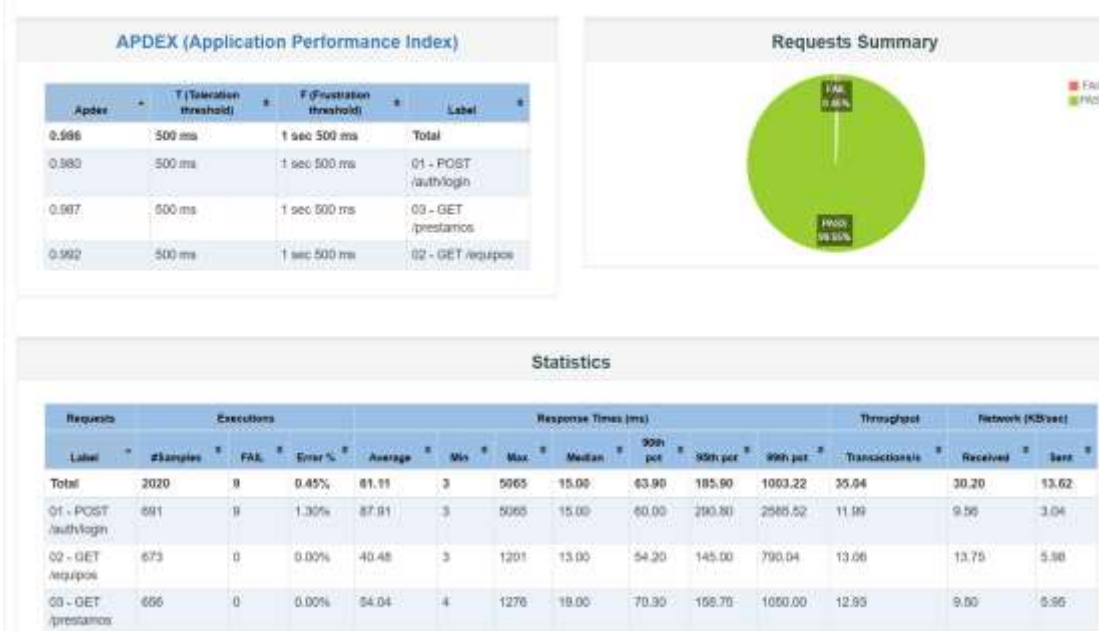


Ilustración 35 Resultados Jmeter

Conclusiones

La construcción y validación del Sistema de Gestión de Préstamos de Laboratorio (SGPLab) demostró que la implementación temprana de estrategias de Aseguramiento de Calidad (QA)

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"

reduce drásticamente la propagación de defectos hacia entornos de producción. La adopción de un modelo de pruebas integral permitió evaluar la aplicación desde múltiples perspectivas, cumpliendo con las características de calidad establecidas por la norma ISO/IEC 25010.

En primer lugar, la exigencia de un 80% de cobertura mediante JUnit y JaCoCo garantizó que la lógica central del negocio —especialmente las transiciones de estado de los equipos y las validaciones de penalizaciones— funcionara correctamente ante escenarios de borde. Por otro lado, la sustitución de metodologías tradicionales de prueba de interfaz por las auditorías automatizadas de **Google Lighthouse** permitió optimizar métricas cruciales de rendimiento web (Core Web Vitals) y accesibilidad, asegurando que el frontend no solo fuera funcional, sino eficiente y centrado en el usuario.

Asimismo, las pruebas de carga con **Apache JMeter** fueron vitales para comprender el comportamiento del sistema bajo concurrencia, estableciendo los límites operativos de la base de datos y la API. Finalmente, la integración de **SonarQube** junto con la infraestructura orquestada en **Docker** y **GitHub Actions** transformó un proyecto académico en un entorno de desarrollo profesional, probando que la automatización del pipeline de CI/CD es indispensable para mantener un ciclo de entrega continuo, predecible y de alta calidad.

Referencias

[1] Spring Framework, "Spring Boot Reference Documentation," *Spring.io*, 2023. [Online]. Disponible en: <https://docs.spring.io/spring-boot/docs/current/reference/html/>



- [2] JUnit Team, "JUnit 5 User Guide," *junit.org*, 2023. [Online]. Disponible en: <https://junit.org/junit5/docs/current/user-guide/>
- [3] Google for Developers, "Lighthouse: Overview," *Chrome Developers*, 2023. [Online]. Disponible en: <https://developer.chrome.com/docs/lighthouse/overview/>
- [4] Apache Software Foundation, "Apache JMeter - User's Manual," *jmeter.apache.org*, 2023. [Online]. Disponible en: <https://jmeter.apache.org/usermanual/index.html>
- [5] SonarSource, "SonarQube Documentation," *docs.sonarsource.com*, 2023. [Online]. Disponible en: <https://docs.sonarsource.com/sonarqube/latest/>
- [6] Docker Inc., "Docker Documentation," *docs.docker.com*, 2023. [Online]. Disponible en: <https://docs.docker.com/>
- [7] Marc R. Hoffmann et al., "JaCoCo Java Code Coverage Library," *eclemma.org*, 2023. [Online]. Disponible en: <https://www.eclemma.org/jacoco/>
- [8] ISO/IEC, "ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models," *International Organization for Standardization*, 2011.

